Pipeline hazards

- There are situations in the pipeline when the next instruction can not execute in the following clock cycle, these events are called hazards and there are three types:
- Structural, data and control hazards.
- Structural Hazard: An occurrence in which planned instruction can not execute in the proper clock cycle because the hardware cannot support the combinations of instructions that are set to execute in the given clock cycle.
- With reference to laundry example the structural hazard would occur if we use the combination of washer-dryer instead of separate washer and dryer, or you roommate was busy doing something else and wouldn't put clothes away.
- Hence, the carefully scheduled pipeline plans would than be foiled.
- If the pipeline in the previous Fig,has fourth instruction then we would see that the first instruction is accessing data from the memory and the fourth instruction is fetching an instruction from that same memory.
- Without two memories our pipeline could have a structural hazard.

* A structural hazard may occur when two instructions have a conflict on the same set of resources in a cycle
* Example :
  * Assume that we have an add instruction that can read one operand from memory
  * add r1, r2, 10[r3]

```
[1]: st r4, 20[r5]
[2]: sub r8, r9, r10
[3]: add r1, r2, 10[r3]
```

* This code will have a structural hazard
  * [3] tries to read 10[r3] (MA unit) in cycle 4
  * [1] tries to write to 20[r5] (MA unit) in cycle 4
* Does not happen in our pipeline

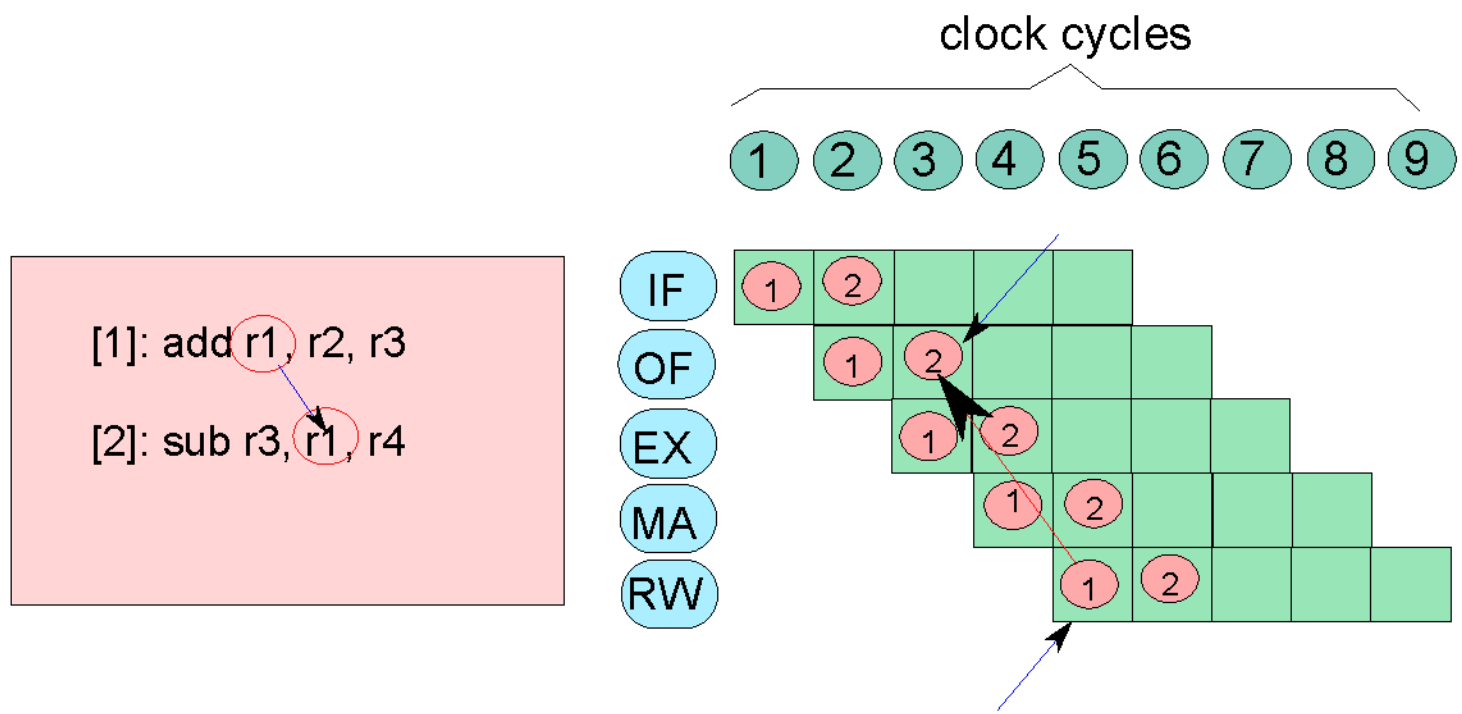
- Data Hazard: An occurrence in which a planned instruction can not execute in the proper clock cycle because the data that is needed to execute the instruction is not yet available.
- Occurs when the pipeline must be stalled because one step must wait for another to complete.
- In the computer pipeline, data hazards arise from the dependence of one instruction on an earlier one that is still in the pipeline(relationship that doesn't really exists when doing laundry).

add $s0 ,$t0,$t1

sub $t2,$s0,$t3

Data hazard could severely stall the pipeline, the add instruction does not write its result until the fifth stage, meaning that we would have to add three bubbles to the pipeline.

Data Hazard



* Instruction 2 will read incorrect values !!!

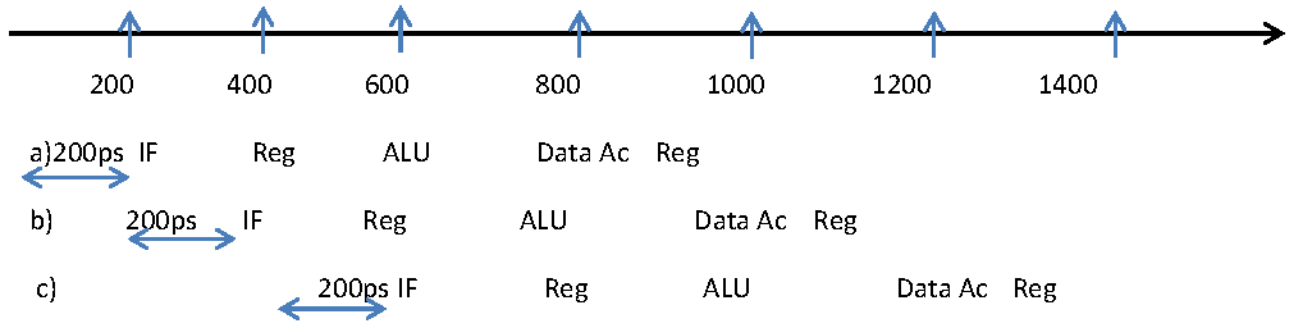
Definition 61

A hazard is defined as the possibility of erroneous execution of an instruction in a pipeline. A data hazard *represents the possibility of erroneous execution because of the unavailability, or availability of incorrect data.*

*This situation represents a data hazard
*In specific,
    *it is a RAW (read after write) hazard
*The earliest we can dispatch instruction 2, is cycle 5

- Although we would try to rely on compilers to remove all such hazards, the result would not be satisfactory. These dependences happen just too often and the delay is just too long to expect the compiler to rescue us from this dilemma.
- Forwarding or bypassing:
- For the code sequence mentioned earlier,as soon as the ALU creates the sum for add, we can supply it as input for the subtract.Adding extra h/w to retrieve the missing item early from the internal resources is called forwarding or bypassing.

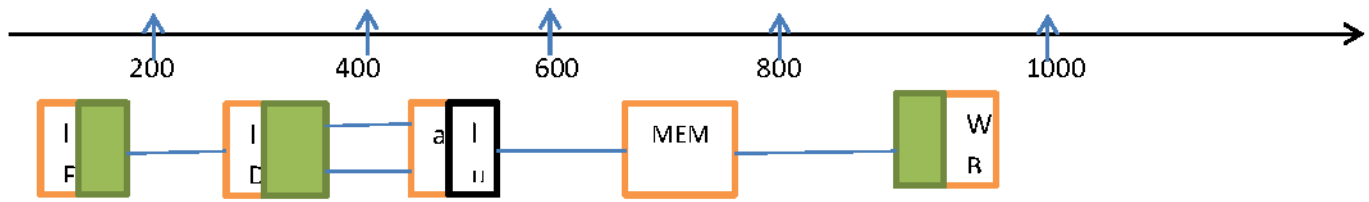
Graphical representation
Fig.2 add $s0,$t0,$t1



a)lw $1,100($0)

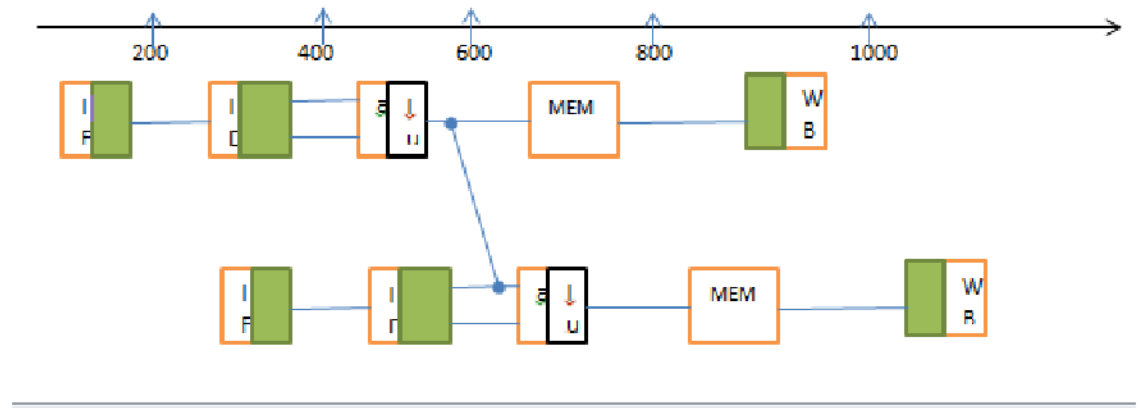
b)lw $2,200($0)

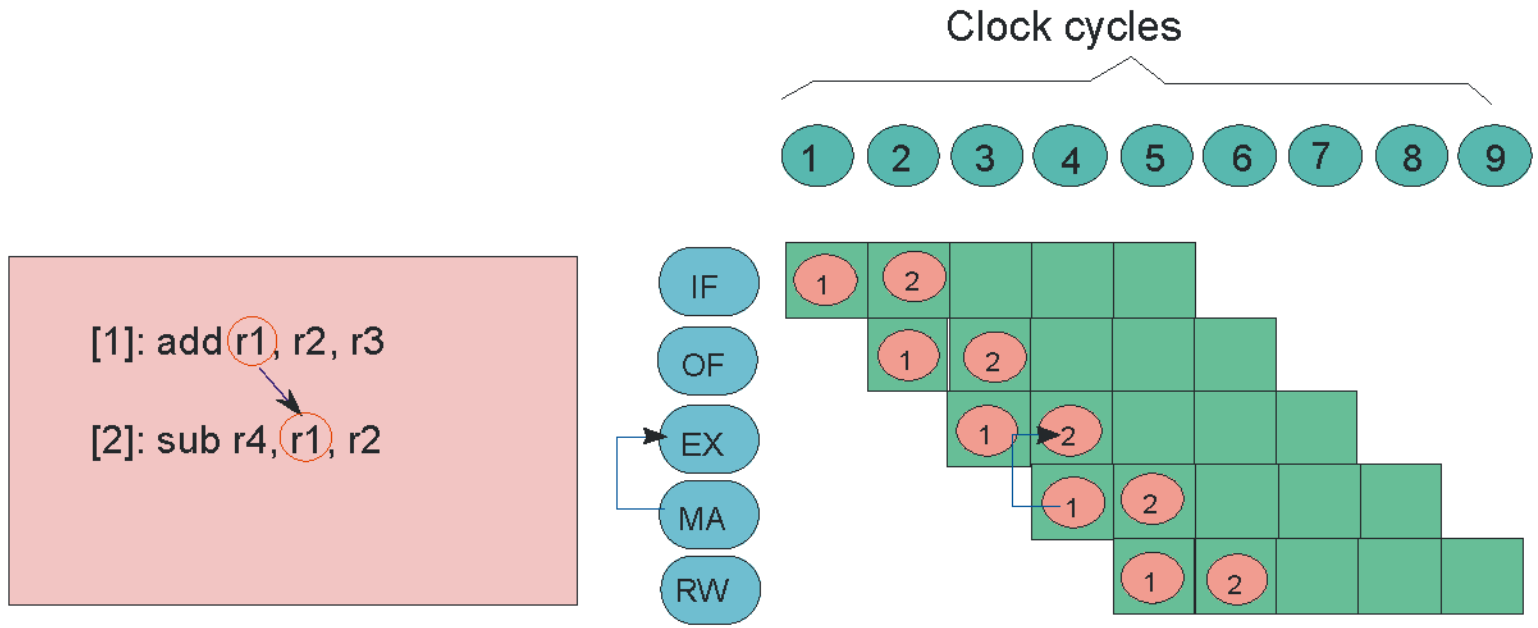
c)lw $3,300($0)



Forwarding

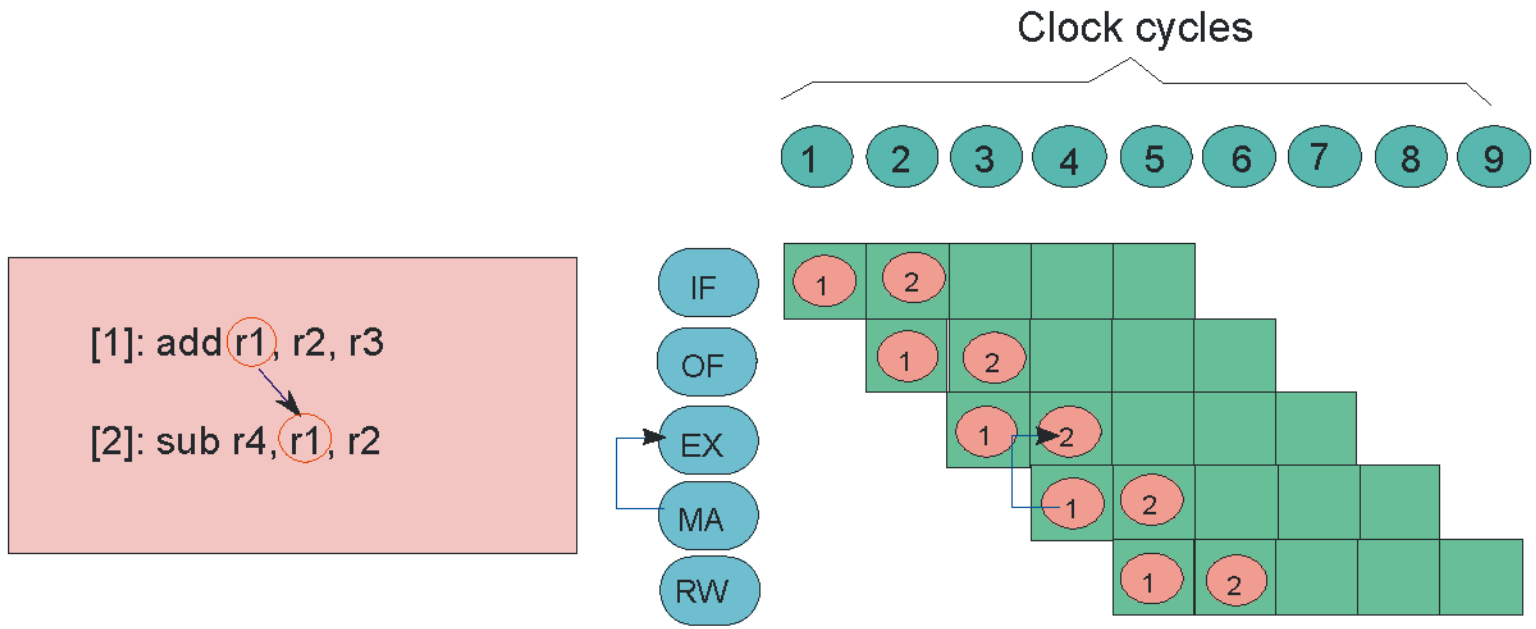
- add $s0,$t0,$t1
- sub $t2,$s0,$t3





- If the correct value is already there in another stage, we can **forward** it.

Forwarding from MA to EX



* Fowarding in cycle 4 from instruction [1] to [2]

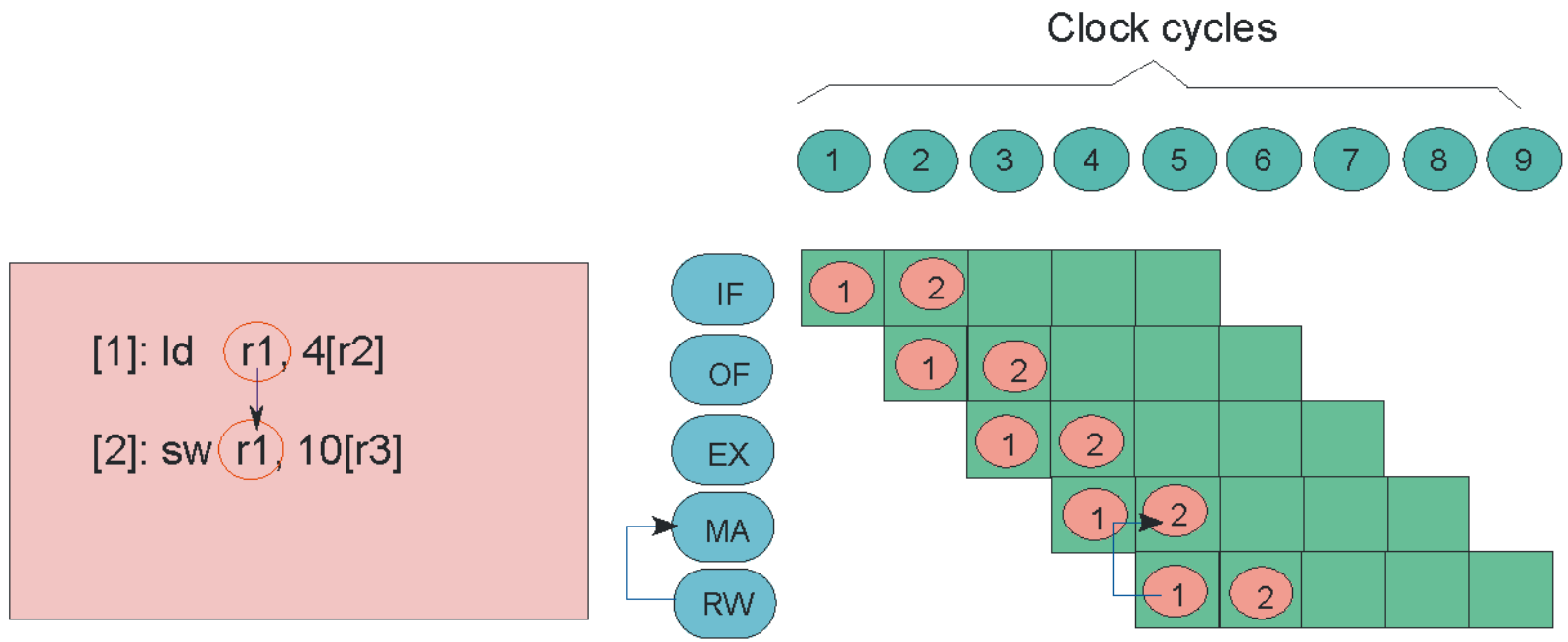
Different Forwarding Paths

* We need to add a multitude of forwarding paths
* Rules for creating forwarding paths
  * Add a path from a later stage to an earlier stage
  * Try to add a forwarding path as late as possible. For, example, we avoid the EX → OF forwarding path, since we have the MA → EX forwarding path
  * The IF stage is not a part of any forwarding path.

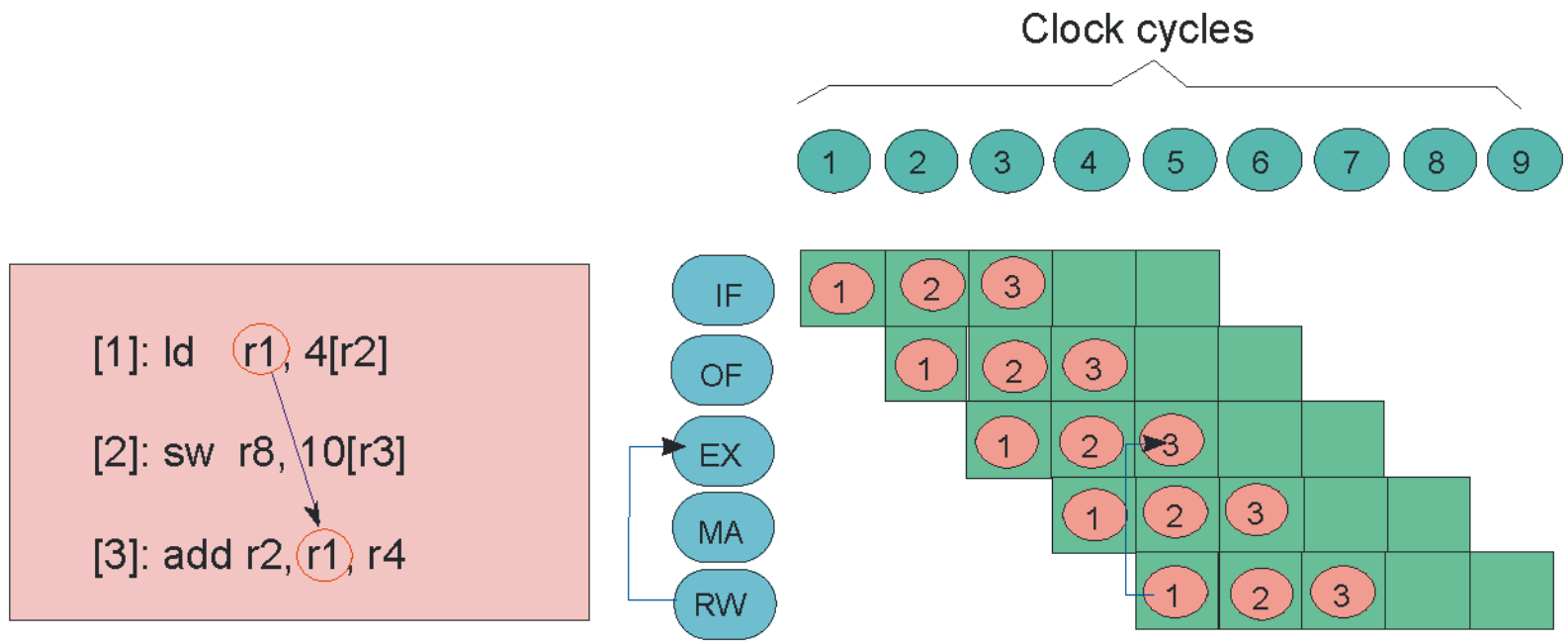
Forwarding Path

* 3 Stage Paths
  * RW → OF
* 2 Stage Paths
  * RW → EX
  * MA → OF (**X** Not Required)
* 1 Stage Paths
  * RW → MA (load to store)
  * MA → EX (ALU Instructions, load, store)
  * EX → OF (**X** Not Required)

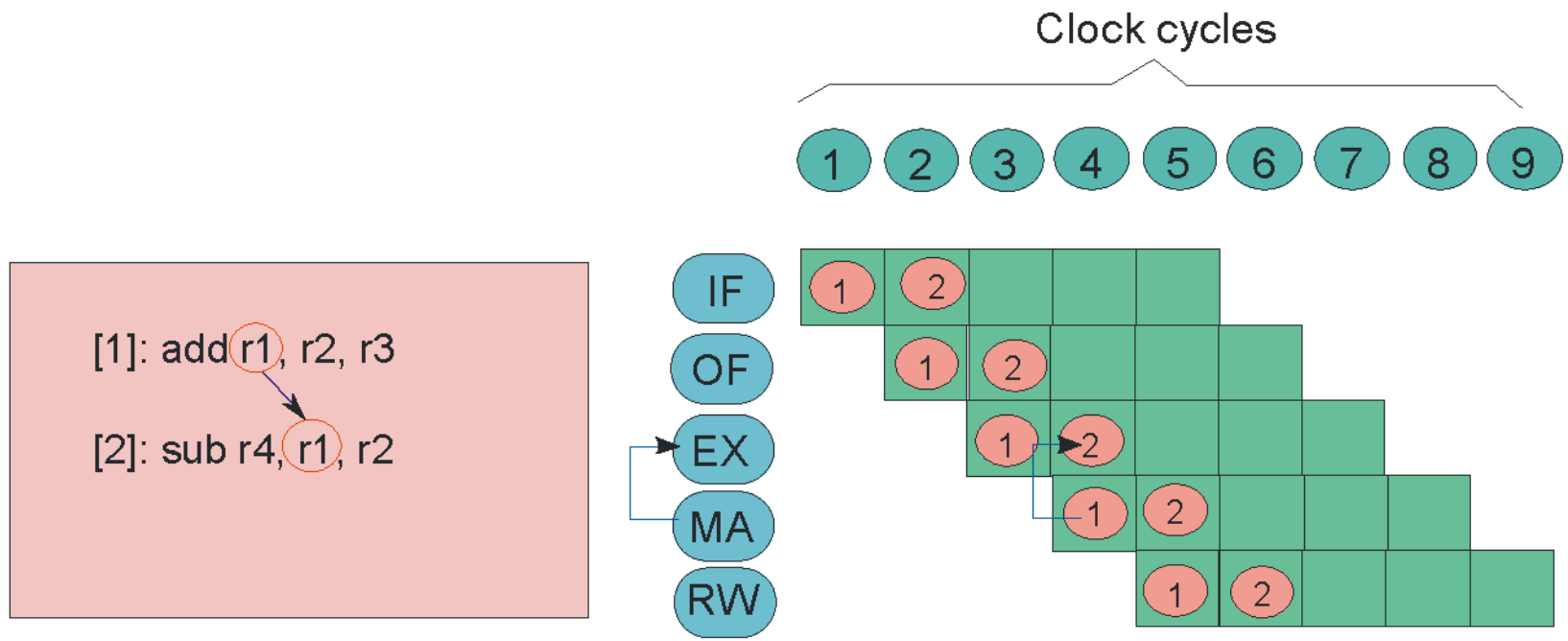
Forwarding Paths : RW → MA



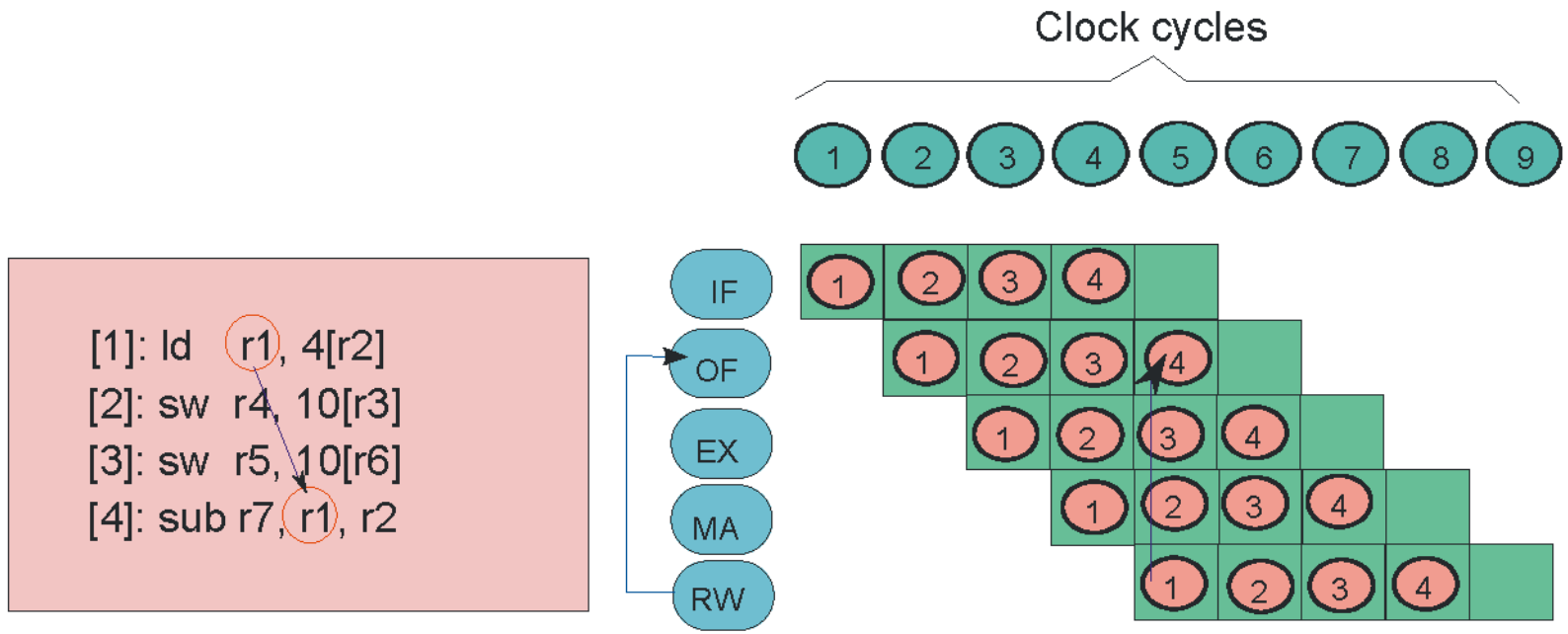
Forwarding Paths : RW → EX



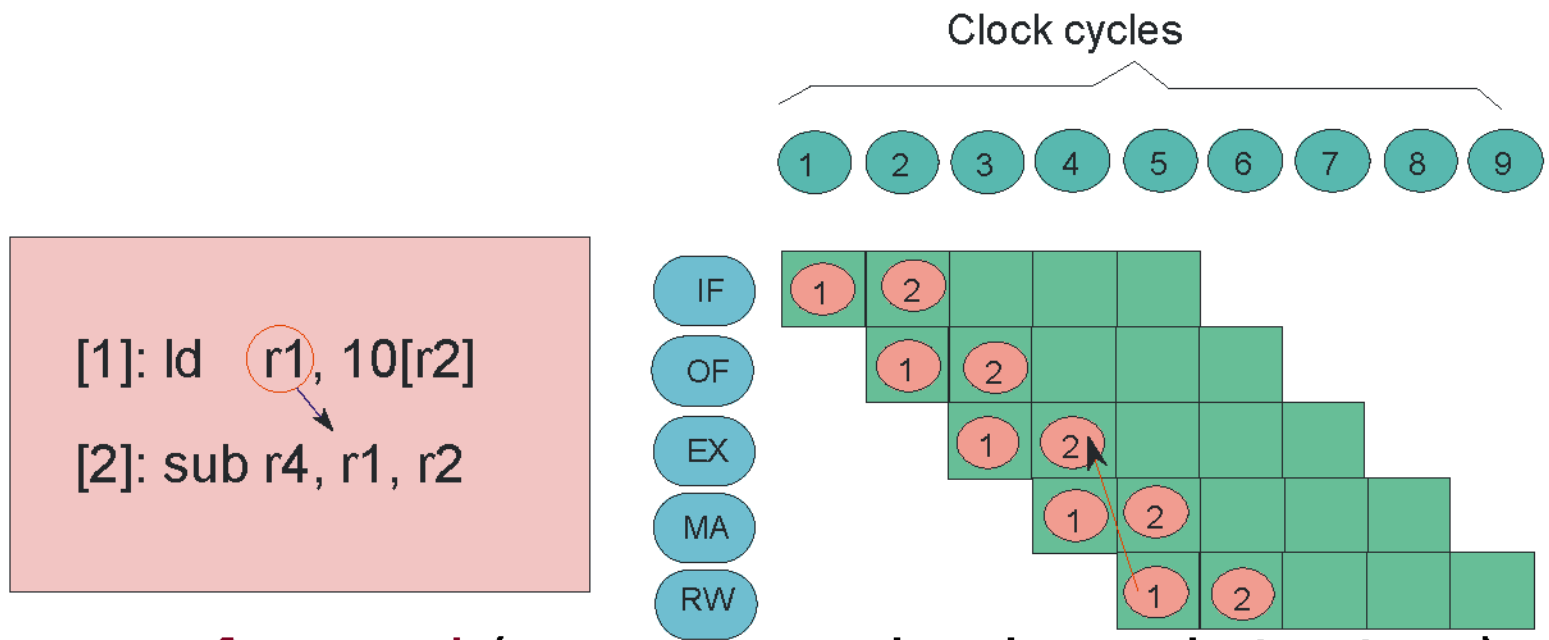
Forwarding Path : MA → EX



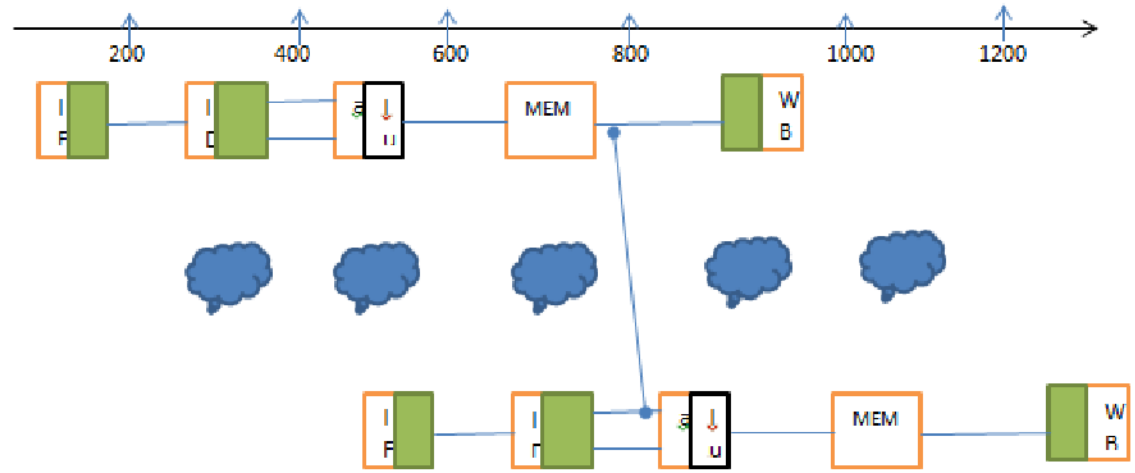
Forwarding Path : RW → OF



- Forwarding works very well but cannot prevent all pipeline stalls.
- Suppose the first instruction were load of $s0,instead of an add, from the figure it is clear that the desired data would be available only after the fourth stage of the first instruction in the dependence,which is too late for the input of the third stage of sub.'
- Even,with forwarding,we would have to stall one stage for a load use data hazard(specific form of data hazard in which the data requested by a load instruction has not yet become available when it is requested)



* Cannot forward (arrow goes backwards in time)
* Need to add a bubble (then use RW → EX forwarding)



lw $s0,20($t1)

sub $t2,$s0,$t3

- We need to stall even with forwarding when an R-format instruction following a load tries to use data.
- A stall initiated in order to resolve a hazard(pipeline stall)

Data Hazards with Forwarding

* Forwarding has unfortunately not eliminated all data hazards
* We are left with one special case.
* Load-use hazard
  * The instruction immediately after a load instruction has a RAW dependence with it.

Consider the c segment: A=B+E and C=B+F,in MIPS

```
lw $t1,0($t0)
lw $t2,4($t0)
add $t3,$t1,$t2
sw $t3,12($t0)
lw $t4,8($01)
add$t5,$t1,$t4
sw $t5,16($t0)
```

Find the hazards in instructions and reorder to avoid any pipeline stalls.

Both `add` instructions have a hazard because of their respective dependence on the immediately preceding `lw` instruction. Notice that bypassing eliminates several other potential hazards including the dependence of the first `add` on the first `lw` and any hazards for store instructions. Moving up the third `lw` instruction eliminates both hazards:

```
lw      $t1, 0($t0)
lw      $t2, 4($t1)
lw      $t4, 8($01)
add     $t3, $t1,$t2
sw      $t3, 12($t0)
add     $t5, $t1,$t4
sw      $t5, 16($t0)
```

On a pipelined processor with forwarding, the reordered sequence will complete in two fewer cycles than the original version.

Control Hazards

- Also called as branch hazard,An occurrence in which the proper instruction can not execute in the proper clock cycle because the instruction that was fetched is not the one that is needed, i.e the flow of instruction addresses is not what pipeline expected.

Suppose our laundry crew was given the happy task of cleaning the uniforms of a football team. Given how filthy the laundry is, we need to determine whether the detergent and water temperature setting we select is strong enough to get the uniforms clean but not so strong that the uniforms wear out sooner. In our laundry pipeline, we have to wait until the second stage to examine the dry uniform to see if we need to change the washer setup or not. What to do?

Here is the first of two solutions to control hazards in the laundry room and its computer equivalent.

Stall: Just operate sequentially until the first batch is dry and then repeat until you have the right formula. This conservative option certainly works, but it is slow.